

StudyFinder Phase 1 Core Functionalities

1. Introduction

StudyFinder is a web application that helps students find and join study groups for their courses. Phase 1 provides the basic features required for a working application.

The Phase 1 core functionalities are:

- User registration
- User login
- Searching for study groups
- Creating a study group
- Viewing study-group information
- Joining a study group
- Leaving a study group
- Connecting the frontend to the backend

The backend was developed using FastAPI, SQLAlchemy, Pydantic, and a relational database. The frontend uses HTML, CSS, and JavaScript.

2. User Registration and Login Schemas

The following schemas validate the information submitted during registration and login.

`EmailStr` ensures that users enter a valid email address. Passwords are accepted during registration and login but are not returned in API responses.

from pydantic import BaseModel, ConfigDict, EmailStr

```
class UserRegister(BaseModel):
    display_name: str
    email: EmailStr
    password: str
    major: str | None = None
    school_year: str | None = None
```

```
class UserLogin(BaseModel):
    email: EmailStr
    password: str
```

```
class AuthUserResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    display_name: str
    email: EmailStr
    major: str | None = None
    school_year: str | None = None
```

```
class AuthResponse(BaseModel):
    message: str
    user: AuthUserResponse
```

Description

UserRegister contains the information needed to create an account. The user must provide a display name, email address, and password. Major and school year are optional.

UserLogin contains the email address and password used to sign in.

AuthUserResponse controls which user information is returned to the frontend. It does not expose the password or password hash.

3. User Registration

The registration endpoint checks whether an account already exists. If the email is available, the password is hashed before the user is stored in the database.

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
```

```
from database import get_db
from models import User
from schemas import AuthResponse, UserRegister
```

```
router = APIRouter()
```

```
@router.post(
    "/register",
    response_model=AuthResponse,
    status_code=status.HTTP_201_CREATED
)
```

```
def register_user(
    user_data: UserRegister,
    db: Session = Depends(get_db)
):
    existing_user = (
        db.query(User)
        .filter(User.email == user_data.email)
        .first()
    )
```

```
    if existing_user:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="A user with this email already exists."
        )
```

```
    password_hash = hash_password(user_data.password)
```

```
    new_user = User(
        display_name=user_data.display_name,
        email=user_data.email,
        password_hash=password_hash,
        major=user_data.major,
        school_year=user_data.school_year
    )
```

```
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
```

```

return {
    "message": "Account created successfully.",
    "user": new_user
}

```

Description

The endpoint receives registration information from the frontend. It searches the database for the submitted email address.

If the email already belongs to another account, the endpoint returns a conflict response with the message "A user with this email already exists."

If the email is available, the password is hashed and the new user is saved. The API then returns the new user's safe account information.

4. User Login

The login endpoint finds the user by email and verifies the submitted password. Invalid credentials return a **401 Unauthorized** response

```

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

```

```

from database import get_db
from models import User
from schemas import AuthResponse, UserLogin

```

```

router = APIRouter()

```

```

@router.post(
    "/login",
    response_model=AuthResponse
)

```

```

def login_user(
    login_data: UserLogin,
    db: Session = Depends(get_db)
):
    user = (
        db.query(User)
        .filter(User.email == login_data.email)
        .first()
    )

```

```

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password."
        )

```

```

    password_is_valid = verify_password(
        login_data.password,
        user.password_hash
    )

```

```

    if not password_is_valid:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password."
        )

    return {
        "message": "Login successful.",
        "user": user
    }

```

Description

The endpoint first searches for an account matching the submitted email address. It then compares the submitted password with the stored password hash.

The same error message is returned for an incorrect email or password. This prevents the API from revealing whether a specific email address is registered.

After a successful login, the frontend stores the returned user information and loads the study-group page.

5. Study Group Schemas

These schemas validate data used when creating, updating, and displaying study groups.

```

from datetime import datetime

```

```

from pydantic import BaseModel, ConfigDict

```

```

class StudyGroupCreate(BaseModel):
    group_name: str
    course_code: str
    description: str | None = None
    creator_user_id: int
    status: str = "open"
    max_members: int = 10

```

```

class StudyGroupUpdate(BaseModel):
    group_name: str
    course_code: str
    description: str | None = None
    status: str = "open"
    max_members: int = 10

```

```

class StudyGroupResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

```

```

    id: int
    group_name: str
    course_code: str
    description: str | None = None
    creator_user_id: int

```

```
status: str
max_members: int
created_at: datetime
```

Description

StudyGroupCreate requires a group name, course code, and creator ID. A group is open by default and has a default limit of ten members.

StudyGroupUpdate represents information that can be changed after a group is created.

StudyGroupResponse defines the group information returned by the API.

6. Creating a Study Group

The create-group endpoint verifies the creator and validates the maximum number of members before saving the group.

```
@router.post(
    "/",
    response_model=StudyGroupResponse,
    status_code=status.HTTP_201_CREATED
)
def create_study_group(
    group_data: StudyGroupCreate,
    db: Session = Depends(get_db)
):
    creator = (
        db.query(User)
        .filter(User.id == group_data.creator_user_id)
        .first()
    )

    if not creator:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Creator user not found."
        )

    if group_data.max_members <= 0:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Maximum members must be greater than zero."
        )

    new_group = StudyGroup(
        group_name=group_data.group_name,
        course_code=group_data.course_code.upper(),
        description=group_data.description,
        creator_user_id=group_data.creator_user_id,
        status=group_data.status,
        max_members=group_data.max_members
    )

    db.add(new_group)
    db.commit()
```

```
db.refresh(new_group)
```

```
return new_group
```

Description

The endpoint checks that the creator's user ID exists. It also prevents groups from being created with a member limit of zero or less.

The course code is converted to uppercase so that course codes are stored consistently. After validation, the group is added to the database and returned to the frontend.

7. Searching for Study Groups

The study-group search endpoint accepts an optional course-code query. It performs a case-insensitive partial search.

```
from fastapi import Depends, Query
from sqlalchemy.orm import Session
```

```
from database import get_db
from models import StudyGroup
from schemas import StudyGroupResponse
```

```
@router.get(
    "/",
    response_model=list[StudyGroupResponse]
)
def get_study_groups(
    course_code: str | None = Query(default=None),
    db: Session = Depends(get_db)
):
    query = db.query(StudyGroup)

    if course_code:
        query = query.filter(
            StudyGroup.course_code.ilike(
                f"%{course_code}%"
            )
        )

    groups = (
        query
        .order_by(StudyGroup.id)
        .all()
    )

    return groups
```

Description

If no course code is provided, the endpoint returns all study groups. If a course code is provided, it filters the groups using a case-insensitive search.

For example, searching for ITSC can return groups with course codes such as ITSC 3155 and ITSC 2214.

8. Viewing One Study Group

The group-details endpoint retrieves a study group using its unique ID.

```
@router.get(
   ("/{group_id}",
    response_model=StudyGroupResponse
)
def get_study_group(
    group_id: int,
    db: Session = Depends(get_db)
):
    group = (
        db.query(StudyGroup)
        .filter(StudyGroup.id == group_id)
        .first()
    )

    if not group:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Study group not found."
        )

    return group
```

Description

The frontend calls this endpoint when the user selects a study group. If the requested group does not exist, the API returns a 404 Not Found response.

If the group exists, its name, course code, description, status, creator, and maximum number of members are returned.

9. Joining a Study Group

The join endpoint verifies the group, user, existing membership, and available group capacity.

```
@router.post(
   ("/{group_id}/members/{user_id}",
    status_code=status.HTTP_201_CREATED
)
def join_study_group(
    group_id: int,
    user_id: int,
    db: Session = Depends(get_db)
):
    group = (
        db.query(StudyGroup)
        .filter(StudyGroup.id == group_id)
        .first()
    )

    if not group:
```

```
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="Study group not found."
    )
```

```
    user = (
        db.query(User)
        .filter(User.id == user_id)
        .first()
    )
```

```
    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User not found."
        )
```

```
    existing_member = (
        db.query(StudyGroupMember)
        .filter(
            StudyGroupMember.group_id == group_id,
            StudyGroupMember.user_id == user_id
        )
        .first()
    )
```

```
    if existing_member:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="User is already a member of this group."
        )
```

```
    member_count = (
        db.query(func.count(StudyGroupMember.user_id))
        .filter(StudyGroupMember.group_id == group_id)
        .scalar()
    )
```

```
    if member_count >= group.max_members:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="This study group is full."
        )
```

```
    new_member = StudyGroupMember(
        group_id=group_id,
        user_id=user_id
    )
```

```
    db.add(new_member)
    db.commit()
    db.refresh(new_member)
```

```
    return {
        "group_id": group_id,
        "user_id": user_id,
```



```
    "message": "User joined the study group."
}
```

Description

A user cannot join a group that does not exist. The endpoint also checks that the user exists and is not already a member.

The current number of members is compared with the group's maximum capacity. If space is available, a new membership record connects the user to the study group.

10. Viewing Group Members

The group-member endpoint returns the users who have joined a particular group.

```
@router.get(
   ("/{group_id}/members",
    response_model=list[StudyGroupMemberResponse]
)
def get_group_members(
    group_id: int,
    db: Session = Depends(get_db)
):
    group = (
        db.query(StudyGroup)
        .filter(StudyGroup.id == group_id)
        .first()
    )

    if not group:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Study group not found."
        )

    members = (
        db.query(
            User.id.label("user_id"),
            User.display_name,
            User.email,
            StudyGroupMember.joined_at
        )
        .join(
            StudyGroupMember,
            User.id == StudyGroupMember.user_id
        )
        .filter(
            StudyGroupMember.group_id == group_id
        )
        .order_by(
            StudyGroupMember.joined_at
        )
        .all()
    )
```

```
return members
```

Description

This endpoint first confirms that the study group exists. It then joins the user and membership tables to retrieve each member's ID, name, email address, and join date.

The result is also used to determine whether the currently logged-in user should see a Join group or Leave group button.

11. Leaving a study Group

The leave-group endpoint finds and deletes the membership connecting a user to a group.

```
@router.delete(
   ("/{group_id}/members/{user_id}",
    status_code=status.HTTP_204_NO_CONTENT
)
def leave_study_group(
    group_id: int,
    user_id: int,
    db: Session = Depends(get_db)
):
    member = (
        db.query(StudyGroupMember)
        .filter(
            StudyGroupMember.group_id == group_id,
            StudyGroupMember.user_id == user_id
        )
        .first()
    )

    if not member:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User is not a member of this group."
        )

    db.delete(member)
    db.commit()

    return None
```

Description

A user can leave a group only if a matching membership exists. If the user is not a member, the endpoint returns a 404 Not Found response.

When the membership exists, it is deleted and the endpoint returns a 204 No Content response.

12. The frontend sends registration and login requests to the authentication endpoints.

```
loginForm.addEventListener("submit", async (event) => {
    event.preventDefault();

    const email = loginEmail.value.trim();
    const password = loginPassword.value;
```

```

    try {
      const response = await fetch(
        `${API_URL}/auth/login`,
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            email,
            password
          })
        }
      );

      const data = await response.json();

      if (!response.ok) {
        throw new Error(
          data.detail || "Login failed."
        );
      }

      currentUser = data.user;

      sessionStorage.setItem(
        "studyfinder-user",
        JSON.stringify(currentUser)
      );

      showApplication();
      loadGroups();
    } catch (error) {
      loginMessage.textContent = error.message;
    }
  });

```

Registration Request

```

registerForm.addEventListener("submit", async (event) => {
  event.preventDefault();

```

```

    const displayName = registerName.value.trim();
    const email = registerEmail.value.trim();
    const password = registerPassword.value;

```

```

    try {
      const response = await fetch(
        `${API_URL}/auth/register`,
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({

```

```

        display_name: displayName,
        email,
        password
    })
  }
);

const data = await response.json();

if (!response.ok) {
  throw new Error(
    data.detail || "Registration failed."
  );
}

currentUser = data.user;

sessionStorage.setItem(
  "studyfinder-user",
  JSON.stringify(currentUser)
);

showApplication();
loadGroups();
} catch (error) {
  registerMessage.textContent = error.message;
}
});

```

Description

The frontend converts the form fields into JSON and sends them to the API. Successful account information is stored in sessionStorage.

After registration or login succeeds, the authentication screen is hidden and the main study-group interface is displayed.

13. Frontend Group Search

The search form sends the entered course code to the backend and displays the matching groups.

```

searchForm.addEventListener("submit", async (event) => {
  event.preventDefault();

  const courseCode = searchInput.value.trim();

  if (!courseCode) {
    searchMessage.textContent =
      "Enter a course code.";
    return;
  }

  searchMessage.textContent = "";

  try {
    const response = await fetch(

```

```

        `${API_URL}/study-groups/?course_code=` +
        encodeURIComponent(courseCode)
    );

    const groups = await response.json();

    if (!response.ok) {
        throw new Error(
            groups.detail || "Search failed."
        );
    }

    displayGroups(groups);
} catch (error) {
    searchMessage.textContent = error.message;
}
});

function displayGroups(groups) {
    groupResults.innerHTML = "";
    resultCount.textContent =
        `${groups.length} groups`;

    if (groups.length === 0) {
        groupResults.innerHTML =
            "<p>No study groups were found.</p>";
        return;
    }

    groups.forEach((group) => {
        createGroupCard(group);
    });
}

```

Description

The search value is URL encoded before it is added to the request. This allows spaces and other characters in course codes to be transmitted correctly.

The results area is cleared before new results are displayed. If no matching groups are found, the page displays an appropriate message.

14. Frontend Group Creation

The create-group form sends the new group information and the logged-in user's ID to the backend.

```

createGroupForm.addEventListener(
    "submit",
    async (event) => {
        event.preventDefault();

        const groupData = {
            group_name: groupName.value.trim(),
            course_code: courseCode.value.trim(),
            description: groupDescription.value.trim(),
            creator_user_id: currentUser.id,

```

```

        status: "open",
        max_members: 10
    };

    try {
        const response = await fetch(
            `${API_URL}/study-groups/`,
            {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(groupData)
            }
        );

        const data = await response.json();

        if (!response.ok) {
            throw new Error(
                data.detail ||
                "Unable to create group."
            );
        }

        createGroupForm.reset();
        createGroupDialog.close();
        showNotification("Study group created.");
        loadGroups();
    } catch (error) {
        createGroupMessage.textContent =
            error.message;
    }
}
);

```

Description

The logged-in user becomes the creator of the new study group. The form collects the group name, course code, and description.

After the group is successfully created, the form is cleared, the dialog closes, and the group list is refreshed.

Frontend Join and Leave Operations

The frontend selects the proper HTTP method based on whether the user is currently a member.

```

async function updateMembership(groupId, isMember) {
    if (!currentUser) {
        showNotification("Please sign in first.");
        return;
    }

```

```

    const method = isMember ? "DELETE" : "POST";

```

```

    const url =
        `${API_URL}/study-groups/${groupId}` +

```

```

    `/members/${currentUser.id}`;

    try {
      const response = await fetch(url, {
        method
      });

      if (!response.ok) {
        const data = await response.json();

        throw new Error(
          data.detail ||
          "Unable to update membership."
        );
      }

      if (isMember) {
        showNotification(
          "You left the study group."
        );
      } else {
        showNotification(
          "You joined the study group."
        );
      }

      await openGroupDetails(groupId);
      await loadGroups();
    } catch (error) {
      showNotification(error.message);
    }
  }
}

```

A POST request joins a group, while a DELETE request leaves a group. The group ID and current user ID are included in the URL.

After the request succeeds, the group details and search results are refreshed so the interface shows the current membership information.

16. Api Router Config

The FastAPI application connects each Phase 1 router to its correct URL prefix.

```

from fastapi import FastAPI

from routers import auth
from routers import group_members
from routers import study_groups
from routers import users

app = FastAPI(
    title="StudyFinder API"
)

@app.get("/")
def root():

```

```
    return {  
        "message": "StudyFinder API is running"  
    }  
}
```

```
app.include_router(  
    users.router,  
    prefix="/users",  
    tags=["Users"]  
)
```

```
app.include_router(  
    study_groups.router,  
    prefix="/study-groups",  
    tags=["Study Groups"]  
)
```

```
app.include_router(  
    group_members.router,  
    prefix="/study-groups",  
    tags=["Group Members"]  
)
```

```
app.include_router(  
    auth.router,  
    prefix="/auth",  
    tags=["Auth"]  
)
```

Description

The router configuration separates the backend into smaller modules. Authentication operations use the /auth prefix. Study-group and membership operations use the

/study-groups prefix.

The root endpoint provides a simple way to confirm that the deployed API is running.

Summary

The Phase 1 implementation provides the basic StudyFinder workflow. A new user can register an account, log in, search for a course, create a study group, view group information, join a group, and leave a group.

Input validation and API error responses prevent common problems such as duplicate accounts, nonexistent users, nonexistent groups, repeated memberships, and joining a full group. The frontend communicates with the FastAPI backend using JSON requests and updates the interface after every successful operation.

